Ingeniería de Software

Seminario de Arquitecturas de Software: Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Equipo: Merxirve

Integrantes (C311):

- Enrique González González — C311
- Ernesto Javier Govea Varona — C311
- Olivia Ortiz Arboláez — C311
- Anthony Cruz Garcia — C311
- Jose Manuel Porras Guerra — C311

Pregunta	Responsable
1	*[nombre del assignee]*
2	Olivia Ortiz Arboláez
3	*[nombre del assignee]*
4	*[nombre del assignee]*
5	*[nombre del assignee]*
6	*[nombre del assignee]*

Repositorio del proyecto:
https://github.com/Anthonycg2003/seminario-IS-Arquitecturas-de-Software

Índice

1. Análisis de trade-offs **2**
1.1. Enunciado 2
1.2. Respuesta 2

2. Hexagonal vs. Onion: simetría y jerarquía de capas **2**
2.1. Enunciado 2
2.2. Respuesta 2

3. DDD y contextos delimitados en un dominio de seguros **4**
3.1. Enunciado 4
3.2. Respuesta 5

4. Inversión de dependencias en la práctica **5**
4.1. Enunciado 5
4.2. Respuesta 5

5. Kitty Problem y principio abierto/cerrado **6**
5.1. Enunciado 6
5.2. Respuesta 6

6. N-Capas y atributos de calidad **6**
6.1. Enunciado 6
6.2. Respuesta 6

Referencias bibliográficas **7**

Declaración de Uso de la Inteligencia Artificial **8**

1. Análisis de trade-offs (N-Capas vs. Clean/Onion/Hexagonal)

1.1. Enunciado

Un equipo de desarrollo está construyendo un sistema de gestión de inventario. Inicialmente, eligen una Arquitectura N-Capas tradicional por su simplicidad y familiaridad. Después de un año, el sistema se ha vuelto un monolito difícil de mantener, donde los cambios en la base de datos afectan la lógica de negocio y la interfaz de usuario, y las pruebas unitarias son lentas y frágiles.

Con base en los principios de la Arquitectura Limpia y la Arquitectura Hexagonal, explique detalladamente qué error fundamental de acoplamiento se está cometiendo y cómo la introducción de la Regla de Dependencia (Martin) o el uso de Puertos y Adaptadores (Cockburn) resolverían estructuralmente este problema. ¿Cuál sería el impacto en la testabilidad y la capacidad de cambiar la base de datos de SQL Server a MongoDB sin afectar la lógica de negocio?

1.2. Respuesta

[Redactar la respuesta con citas, por ejemplo (Cockburn, 2005; Martin, 2017).]

2. Hexagonal vs. Onion: simetría y jerarquía de capas

2.1. Enunciado

La Arquitectura Hexagonal de Cockburn y la Arquitectura Onion de Palermo comparten el objetivo de aislar el núcleo de negocio, pero utilizan representaciones visuales y metáforas radicalmente diferentes (un hexágono simétrico frente a capas concéntricas).

Analice en profundidad qué problema conceptual resuelve la simetría de la representación hexagonal que una representación de capas apiladas (como la de Onion o la N-Capas tradicional) no logra transmitir de forma intuitiva. ¿Cómo se traduce el concepto de «Puerto y Adaptador» en la práctica de definir interfaces en las capas internas de la Arquitectura Onion? ¿Por qué Cockburn afirma que el hexágono no tiene seis lados por una razón numérica, sino para eliminar la asimetría izquierda-derecha o arriba-abajo?

2.2. Respuesta

La metáfora visual elegida para representar una arquitectura condiciona profundamente la perspectiva de los desarrolladores respecto a las dependencias, el flujo de control y el desacoplamiento técnico (Cockburn, 2005; Martin, 2017; Palermo, 2008). A continuación, se analiza en detalle cada uno de los aspectos solicitados:

1. El problema conceptual que resuelve la simetría de la representación hexagonal

En las arquitecturas tradicionales N-Capas (Presentación arriba → Negocio al centro → Datos abajo), la disposición apilada introduce tres distorsiones conceptuales críticas:

1. **La ilusión de la Base de Datos como fundamento:** Colocar la capa de persistencia en la base del diagrama sugiere erróneamente que la base de datos es el cimiento o pilar sobre el cual descansa todo el software, condicionando el modelado de dominio a esquemas relacionales o tablas de almacenamiento (Cockburn, 2005; Martin, 2017).

2. **La prioridad jerárquica de la UI:** Ubicar la interfaz de usuario en la cima la presenta como el único punto de entrada «natural» del sistema (Cockburn, 2005).

3. **La simetría de la representación hexagonal:** Resuelve este dilema al establecer que, desde la perspectiva del núcleo de la aplicación (*Application Core*), **todos los componentes externos son simétricos e equivalentes** (Cockburn, 2005). No existe distinción funcional o jerárquica entre una interfaz gráfica web, una prueba automatizada xUnit o una petición HTTP enviada por un tercero: todos son actores primarios (*drivers*) que invocan casos de uso. De la misma manera, una base de datos PostgreSQL, un broker de mensajes (como RabbitMQ) o una API externa son receptores/proveedores secundarios (*driven*) (Cockburn, 2005). El hexágono transmite de forma directa la frontera binaria fundamental: **Interior (lógica de negocio pura)** versus **Exterior (detalles tecnológicos y mecanismos de E/S)** (Cockburn, 2005; Martin, 2017).

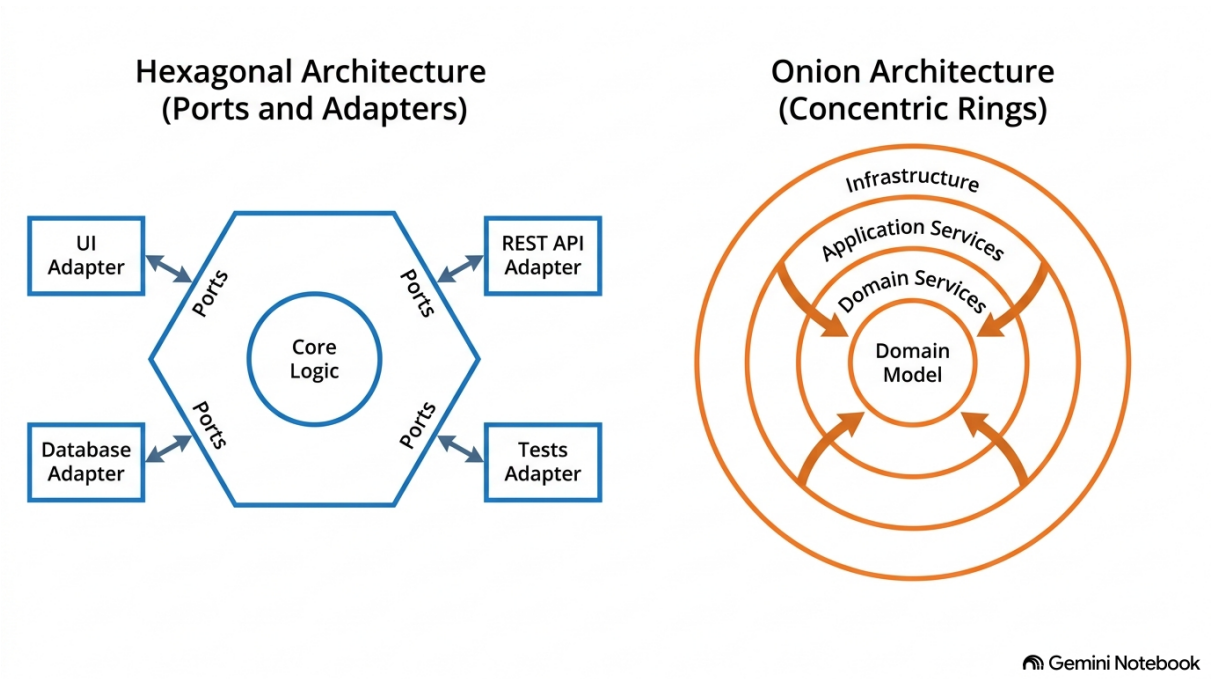


Figura 1: Comparación visual entre la simetría del hexágono y la jerarquía de capas de Onion.

2. Geometría del hexágono: eliminación de ejes de asimetría

Cockburn enfatiza que la elección de una figura geométrica de seis lados **no responde a una razón numérica** (el sistema no está limitado a seis componentes, seis puertos o seis tecnologías) (Cockburn, 2005). El propósito de utilizar un hexágono es

romper deliberadamente con la metáfora unidimensional propia de las capas apiladas («arriba-abajo») o de los diagramas de tuberías («izquierda-derecha») (Cockburn, 2005). Un esquema rectangular tienta visualmente a encasillar «entradas por la izquierda» y «salidas por la derecha». El hexágono proporciona múltiples aristas simétricas e independientes para dibujar y distribuir puertos y adaptadores (UI, DB, servicios REST, tests, mensajes) en diversas direcciones alrededor del núcleo, evidenciando que el dominio no depende de ningún medio de transporte o almacenamiento en particular (Cockburn, 2005; Martin, 2017).

3. Traducción de «Puerto y Adaptador» en las capas internas de Onion

En la Arquitectura Onion de Palermo, el sistema se estructura en capas o anillos concéntricos alrededor del *Domain Model* y los *Domain Services*, los cuales son rodeados por los *Application Services* y la capa externa de *Infrastructure* (Palermo, 2008). El concepto de **Puerto y Adaptador** se traduce operacionalmente en la Arquitectura Onion mediante la aplicación estricta del **Principio de Inversión de Dependencias (DIP)** y la **Regla de Dependencia** (Martin, 2017; Palermo, 2008):

- **Puerto ≡ Interfaz abstracta en capas internas (Core):** En Onion, un *Puerto* es un contrato de interfaz definido en las capas del centro (dentro del *Domain* o *Application Services*) (Cockburn, 2005; Palermo, 2008). Por ejemplo, la interfaz `IOrderRepository` o `IPaymentGateway` se declara dentro del núcleo y especifica *qué* requiere el dominio sin depender de detalles de implementación (Palermo, 2008).
- **Adaptador ≡ Clase concreta en la capa externa (Infrastructure):** Un *Adaptador* es la implementación concreta de dicha interfaz, la cual reside exclusivamente en el anillo periférico de *Infrastructure* o *UI* (Cockburn, 2005; Palermo, 2008). Por ejemplo, `SqlOrderRepository` (que implementa `IOrderRepository`) o `StripeAdapter`.

Dado que en Onion todas las dependencias del código fuente apuntan obligatoriamente hacia el centro (Palermo, 2008), la infraestructura externa depende de la interfaz definida en el núcleo interno. Así, se garantiza la total independencia y testabilidad del dominio frente a cambios tecnológicos (Martin, 2017; Palermo, 2008).

3. DDD y contextos delimitados en un dominio de seguros

3.1. Enunciado

Una empresa de seguros necesita desarrollar un sistema para gestionar pólizas de automóviles. El sistema debe interactuar con un departamento de siniestros, un sistema de facturación de primas y una aplicación para que los ajustadores de seguros evalúen daños en el lugar del accidente. Cada uno de estos subsistemas tiene su propia terminología y preocupaciones (por ejemplo, «póliza» para un actuario es un contrato con cláusulas, para facturación es una fuente de ingresos, y para un ajustador es un número de referencia).

Basado en los conceptos de Domain-Driven Design (Evans) y Bounded Context (Richards & Ford), explique cómo dividiría este sistema en varios contextos delimitados.

¿Cómo evitaría que el modelo de «Póliza» se contamine con detalles de siniestros o facturación? ¿Cómo se comunicarían estos contextos de manera desacoplada? Relacione esta solución con la implementación de una Arquitectura Orientada al Dominio (como Onion o Clean) y la creación de un Adaptador de Servicio para la comunicación externa, tal como se describe en la Guía de Arquitectura N-Capas.

3.2. Respuesta

[Redactar con (De la Torre Llorente et al., 2010; Richards & Ford, 2020).]

4. Inversión de dependencias en la práctica

4.1. Enunciado

Un equipo de desarrollo está trabajando en un sistema de gestión de pedidos. La lógica de negocio principal se encuentra en una clase `OrderProcessor` que, para completar su tarea, necesita comunicarse con un servicio externo de validación de direcciones. La implementación actual tiene una dependencia directa: `OrderProcessor` instancia y llama directamente a la clase `AddressValidatorService`, que a su vez depende de una librería HTTP específica para comunicarse con el servicio externo.

Con base en los principios de la Inversión de Dependencias (Martin) y la Regla de Dependencia (Martin), analice y responda:

- **a)** ¿Por qué esta dependencia directa viola los principios fundamentales de las arquitecturas Hexagonal, Limpia y Onion? Explique específicamente qué dirección deberían tener las dependencias del código fuente y por qué la implementación actual no la cumple.
- **b)** Rediseñe la interacción entre `OrderProcessor` y `AddressValidatorService` para que se alinee con los principios de estas arquitecturas. Defina claramente qué componente (interfaz) debe residir en el núcleo de la aplicación y qué componente debe residir en la capa de infraestructura o en el exterior del hexágono. Justifique su diseño basándose en el principio de que «los detalles deben depender de las abstracciones».
- **c)** Explique cómo este nuevo diseño permite que la lógica de negocio (`OrderProcessor`) sea probada de forma aislada, sin necesidad de realizar llamadas HTTP reales al servicio externo, y sin depender de la librería HTTP subyacente. ¿Qué implicaciones tiene este diseño para la mantenibilidad y la capacidad de cambiar el proveedor de validación de direcciones en el futuro?

4.2. Respuesta

4.2.1. Violación de principios

4.2.2. Rediseño

4.2.3. Testabilidad y evolución

[Fundamentar con (Cockburn, 2005; Martin, 2017; Palermo, 2008).]

5. Kitty Problem y principio abierto/cerrado

5.1. Enunciado

En su libro *Fundamentals of Software Architecture*, Richards y Ford presentan el «Kitty Problem» para ilustrar un fallo común en el diseño de arquitecturas orientadas a servicios. Describa este problema: un requisito transversal (como la entrega de gatitos) afecta a múltiples servicios previamente existentes (TaxiUI, TaxiFinder, TaxiSelector, etc.), requiriendo cambios coordinados en todos ellos.

¿Por qué la solución de dividir el sistema en servicios basados en verbos o entidades funcionales (como TaxiFinder o TaxiSelector) llevó a este problema? Basándose en los principios de DDD y la Arquitectura Onion/Clean, ¿cómo se podría haber diseñado el sistema para que un cambio de este tipo afectara a un solo componente o servicio? Proponga una solución utilizando componentes como Kittens y Rides que se añadan como plugins, explicando cómo esto respeta el Principio de Abierto/Cerrado (OCP) y qué papel juegan los Adaptadores en esta extensión.

5.2. Respuesta

[Redactar con (Martin, 2017; Palermo, 2008; Richards & Ford, 2020).]

6. N-Capas y atributos de calidad

6.1. Enunciado

La Arquitectura N-Capas es criticada por su bajo rendimiento en características como «Agilidad general», «Facilidad de despliegue» y «Escalabilidad» (Richards, 2015). Un estudiante propone que, con el uso de capas abiertas (*open layers*) y técnicas como el «fast-lane reader», se pueden mitigar estos problemas.

Analice esta afirmación. ¿Cómo afecta la introducción de capas abiertas al principio de Aislamiento de Capas y a la mantenibilidad a largo plazo? ¿Por qué la solución de capas abiertas puede llevar a un «sinkhole anti-pattern» y qué dice la regla del 80/20 sobre esto? Finalmente, explique por qué, para un sistema con requisitos de alta escalabilidad y agilidad, una arquitectura como la Hexagonal o la Limpia, que se basan en los principios de DDD y desacoplamiento, son inherentemente más adecuadas que una Arquitectura N-Capas, incluso con optimizaciones.

6.2. Respuesta

[Redactar con (Martin, 2017; Richards, 2015; Richards & Ford, 2020).]

Referencias bibliográficas

Cockburn, A. (2005). *Hexagonal Architecture (Ports and Adapters)* [Consultado para el seminario]. Consultado el 1 de enero de 2026, desde https://alistair.cockburn.us/hexagonal-architecture/

De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., & Calvarro Nelson, J. (2010). *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0 (Borrador)* (inf. téc.). Microsoft Ibérica.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Palermo, J. (2008). *The Onion Architecture*. Consultado el 1 de enero de 2026, desde https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/

Richards, M. (2015). *Software Architecture Patterns*. O'Reilly Media.

Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

Declaración de Uso de la Inteligencia Artificial

El equipo **no utilizó** herramientas de inteligencia artificial para la elaboración de este seminario.